

Distributed SmoteBagging in Apache Spark: A Comparative Study of Local and Global Designs

Abstract - This paper presents and evaluates three distributed SMOTEBagging implementations in Apache Spark, each reflecting a different trade-off between communication overhead and oversampling quality. Variant 1 (V1) performs random shuffle, oversampling, bagging, and model training independently within each partition. Variant 2B (V2B) introduces a single stratified shuffle to guarantee proportional minority representation per partition. Variant 3 (V3) broadcasts the full minority training set to all workers, enabling globally aware SMOTE synthesis while keeping the majority class distributed. All three variants are evaluated on the Kaggle credit card fraud dataset (0.17% minority class) across classification performance, partition count sensitivity, and dataset size scaling experiments. V1 achieves the strongest partition speedup (8.97×) but is most sensitive to partition composition. V2B delivers the highest AUPRC (0.83) through guaranteed minority coverage at moderate shuffle cost. V3 achieves the highest AUC-ROC (0.98) but requires threshold tuning due to poor calibration at the default decision boundary. All variants achieve sub-linear runtime growth with dataset size and maintain stable AUC-ROC across data fractions. These findings establish clear deployment guidance: V1 for communication-constrained settings, V2B for reliable fraud detection, and V3 when ranking quality and a small minority set permits broadcast-based oversampling.

Keywords— Apache Spark, bagging, big data, class imbalance, distributed machine learning, ensemble learning, fraud detection, k-nearest neighbours, oversampling, scalability, SMOTEBagging, stratified partitioning.

I. INTRODUCTION

Class imbalance is a challenge for machine learning, especially where the minority class is rare but most practically relevant. In fraud detection, legitimate transactions far outnumber fraudulent ones, making standard classifiers biased towards the majority class and often missing the events that matter most. Even in large datasets, severe imbalance may remain, and the minority class may still be too sparse for a classifier to learn robust decision boundaries [1], [2].

A popular solution is SMOTEBagging [3], which combines SMOTE with bootstrap aggregation and is one of the more effective ensemble-based techniques for imbalanced classification. However, although straightforward in a sequential setting, SMOTEBagging becomes non-trivial in a distributed framework such as Apache Spark, where minority examples are split across worker nodes and constructing

neighbour relationships may require expensive communication. This leads to a fundamental trade-off: designs that enhance minority-neighbour access may incur shuffle or broadcast costs, while designs that minimise data movement may weaken oversampling quality.

This study examines that trade-off using three Spark-based SMOTEBagging implementations: *Variant 1* performs random shuffle, oversampling, bagging, and model training independently within each partition; *Variant 2* first redistributes training data so that minority examples are more evenly spread across partitions; and *Variant 3* collects the minority training set once, broadcasts it to all workers, and performs oversampling using a global minority pool while keeping the majority class partitioned. Preprocessing, base learner, and evaluation pipelines are kept constant, so differences are mainly ascribed to distributed design decisions. The following research questions guide this study:

- RQ1. How does the minority-handling strategy affect predictive performance, measured using AUC-ROC, AUPRC, F1-score, and G-mean?
- RQ2. How does each design scale as the number of Spark partitions increases?
- RQ3. How does runtime scale with training-set size, and what bottlenecks emerge as the minority pool grows?

To answer these questions, the three variants are implemented in PySpark and evaluated on the Kaggle credit card fraud dataset, in which fraudulent transactions form approximately 0.17% of all records.

II. LITERATURE REVIEW

Class imbalance occurs when one class substantially outnumbers another, biasing standard classifiers toward the majority class [2]. SMOTE, proposed by Chawla et al. [4], addresses this by generating synthetic minority examples via interpolation between a sample and its k-nearest minority neighbours — producing more representative data than simple duplication. Wang and Yao [3] extended this with SMOTEBagging, combining SMOTE with bootstrap aggregation to improve classifier diversity [5]. Galar et al. [2] identify such hybrid ensemble methods as among the most effective for severe imbalance, directly motivating their use here.

The credit card fraud dataset from the Université Libre de Bruxelles [6] is a standard benchmark for imbalanced classification, containing 284,807 transactions with only 492 fraudulent cases (imbalance ratio ~577:1). Its PCA-

transformed features preserve confidentiality while retaining predictive structure, making it well-suited for evaluating scalable, distributed oversampling approaches.

Adapting SMOTE to distributed settings introduces challenges around neighbourhood quality when minority samples are fragmented across partitions. Juez-Gil et al. [7] address this with Approx-SMOTE, using approximate neighbour search to significantly reduce runtime while maintaining classification performance. Sleeman and Krawczyk [8] propose stratified partitioning to improve minority representation before partition-local oversampling; closely related to the strategy in V2B. Sid and Batouche [9] further demonstrate that partition-level training with majority-vote aggregation can effectively handle imbalance at scale, a design principle shared by all three variants. Together, these works motivate the core trade-off explored here: local designs reduce communication overhead but risk poor minority representation, while global designs improve oversampling quality at greater coordination cost [8], [9].

III. METHODOLOGY

A. Dataset Preprocessing

The time column was dropped, and a stratified train/test split (80/20) was performed prior to any scaling or oversampling to prevent data leakage. The Amount feature was standardised using a StandardScaler fitted exclusively to the training set. All the other features were left unscaled as they were already PCA-normalised. The features were then assembled into a single feature vector using VectorAssembler. All variants are built from scratch without MLlib

B. Variant 1: Local Oversampling SMOTE Bagging

A custom SMOTE model was developed using NumPy implementation and applied locally within each partition using Spark's mapPartitions. For each minority sample, the k nearest neighbours ($k=5$) within the partition were identified using pairwise Euclidean distance. A synthetic sample is generated by linear interpolation between a randomly selected minority sample and one of its neighbours. A three-case fallback strategy was also used to handle edge cases, which were no minority samples (partitions reunited as is), fewer than $k+1$ minority samples (random duplication with replacement), and sufficient minority samples (applying the full SMOTE model). The limitation to that the partitions falling into the first two cases produce lower quality synthetic samples.

Bootstrap samples are drawn locally from each oversampled partition. An ensemble of Decision Tree classifiers is trained on those samples. Predictions are aggregated across ensembles using majority voting. The models were evaluated using AUC-ROC, AUPRC, F1-score, G-mean, Precision, and Recall.

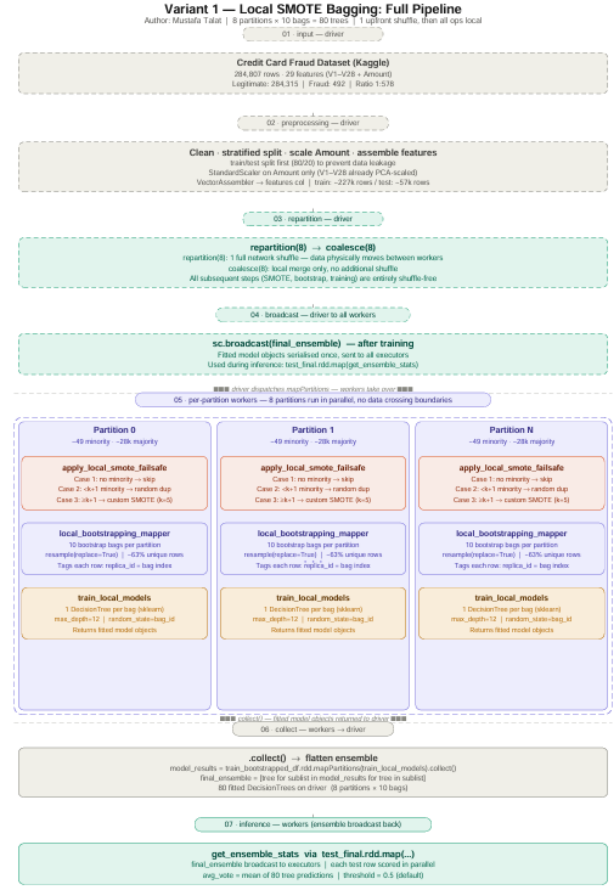


Figure 1: Illustrates the end-to-end data flow for Variant 1

C. Variant 2: Stratified-Repartitioned Local SmoteBagging

After loading from parquet, the default Spark partition layout revealed several partitions containing zero minority or majority samples, a direct consequence of hash-based partitioning, which distributes rows without class-label awareness. With only 394 minority training samples, local SMOTE was entirely unreliable in those partitions.

Variant 2 introduces a stratified repartitioning step before the SMOTEBagging pipeline, ensuring every partition receives a proportional share of both classes. The training data is first filtered into two separate RDDs by class label, *minority_df* and *majority_df*, each inheriting 8 partitions from the earlier coalesce step. Two strategies for redistributing these rows were investigated, referred to as Option A and Option B and illustrated in Figure 2.

In Option A, each row is assigned a random floating-point key derived from a seeded hash of the row itself, with minority and majority RDDs keyed independently using different seeds to avoid interference. These keyed rows are sorted globally across all executors, triggering the first shuffle, during which rows physically move between executors to establish a globally randomised ordering. Each row is then assigned a sequential global index and its target

partition computed as $sequential_index \% P$, with a second shuffle routing each row to its designated partition. Because the global sort randomises row order before index assignment, minority and majority samples are truly interleaved across partitions, guaranteeing a perfectly uniform distribution.

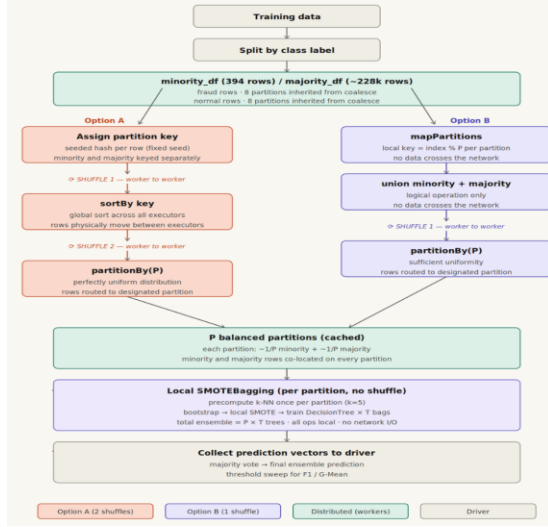


Figure 2: Illustrates the end-to-end data flow for Variant 2

Option B takes a lighter approach. Each partition independently assigns a target key to its own rows using round-robin ($local_index \% P$) with no data crossing the network at this stage. The minority- and majority-keyed RDDs are then combined, a purely logical operation requiring no shuffle, and only a single shuffle occurs when rows are physically routed to their designated partitions. Since the assignment depends on local row ordering within each existing partition rather than a globally randomised sequence, the resulting distribution is slightly less uniform. Therefore, Option A guarantees a more uniform distribution of minorities across partitions, whereas Option B achieves sufficient balance at half the data movement cost. The impact of both strategies on classification performance and scalability is examined in Section IV.

Once repartitioned and cached, the SMOTEBagging pipeline runs entirely within each partition via a single `mapPartitionsWithIndex` call. The test set is collected once and broadcast to all executors, avoiding repeated fetches across the 80 trees. The k-nearest neighbour index is precomputed once per partition using pairwise Euclidean distances and reused across all $T=10$ bags, thereby avoiding redundant distance recomputation. In each bag, local majority rows are bootstrapped with replacement and synthetic minority samples generated using the precomputed k nearest neighbour index, following the balanced-bagging schedule of Wang and Yao [3], where the oversampling rate increases from 10% to 100% across bags. A `DecisionTreeClassifier` is trained per bag and predicts locally on the broadcast test set, returning only an integer prediction vector with no model objects crossing the network. Under $P=8$ partitions and $T=10$

bags, this yields 80 trees in total. The only data crossing the network is the one-time test broadcast and the collection of 80 prediction vectors for driver-side majority vote aggregation.

The primary bottleneck in this pipeline is the upfront stratified shuffle, whose cost grows with training set size. Additionally, the sequential k-nearest neighbour computation within each partition scales quadratically with the local minority sample count, which can become problematic at larger minority set sizes.

D. Variant 3: Global Minority Broadcast SMOTEBagging

Variant 3 exploits a typical asymmetry in extreme-imbalance fraud detection. The entire training set has about 228,000 rows, but only 394 samples in the minority class. This minority set takes up only about 90 KB, which is small enough to materialise once on the driver and broadcast to all executors at a reasonable cost. A diagnostic check of the original partition structure confirmed complete class segregation across partitions, with minority and majority samples residing in entirely separate partitions, making purely local SMOTE unworkable without restructuring. This makes it practical to keep the large majority class distributed while globally replicating the rare but neighbour-sensitive minority class, avoiding both poor minority-neighbour availability within sparse partitions and the need to redistribute the much larger majority class.

After the shared preprocessing stages, the training data are split into two class-specific DataFrames, `train_minority_df` and `train_majority_df`. The minority rows are collected to the driver, converted into a NumPy matrix, and broadcast to all workers using `SparkContext.broadcast`. The majority DataFrame is repartitioned into P partitions and cached. The test set is also collected once and broadcast so that prediction can be performed locally on each worker, avoiding the need to return trained tree objects to the driver.

The main training procedure is implemented using a single `mapPartitionsWithIndex` call over the repartitioned majority RDD. Each worker precomputes the k-nearest-neighbour structure over the full minority set once, then reuses it across all $T=10$ bags, avoiding redundant distance recomputation per bag. In each bag, the worker bootstraps its local majority rows with replacement and constructs the minority side using the balanced-bagging schedule of Wang and Yao [3], varying the proportion of resampled real minority examples and synthetic minority examples across bags. A per-bag oversampling rate $r_b = b/10$ for $b \in \{1, \dots, 10\}$ determines the fraction of the bag filled with resampled *real* minority rows versus synthetic minority rows generated using global minority neighbours. A scikit-learn `DecisionTreeClassifier` is then trained locally on the balanced bag and used to predict on the broad test set. The ensemble therefore contains $P \times T$ trees, and driver-side aggregation is a single majority vote over the collected prediction matrix.

The dominant per-worker preprocessing cost is the k -NN precomputation, with time $O(|G|^2 \cdot F)$ and memory $O(|G|^2)$ for a minority set of size $|G|$ over F features. For the credit-card dataset ($|G| = 394$, $F = 29$), this cost is small enough to incur once per worker before the bag loop. The remaining training cost scales primarily with the size of the local majority partition and the number of bags T . The broadcast payload and quadratic neighbour computation would become bottlenecks if the minority pool grew significantly, in which case a partitioned neighbour search or approximation techniques such as locality-sensitive hashing would be more suitable.

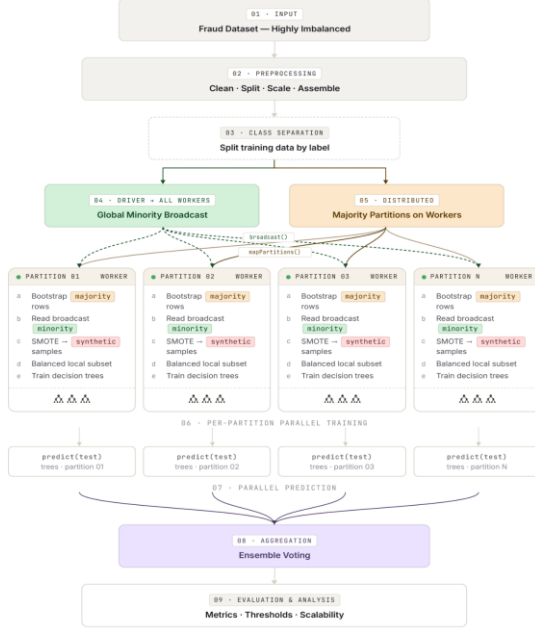


Figure 3: Illustrates the end-to-end data flow for Variant 3.

IV. EXPERIMENTS AND RESULTS

All experiments were conducted on a Databricks cluster running Spark across 8 worker nodes. Each variant trained an ensemble of 80 trees over 8 partitions, with 10 bootstrap bags per partition, a fixed bag size of 10,000 samples, and $k=5$ nearest neighbours for SMOTE. The dataset was split 80/20, yielding 227,464 training samples (384 fraud cases) and 56,959 test samples (108 fraud cases). Given the severe class imbalance (0.172% fraud), AUPRC was treated as the primary evaluation metric alongside Precision, Recall, F1, G-Mean, and AUC-ROC.

A. Classification Performance

Each variant was evaluated on the held-out test set using threshold sensitivity analysis over thresholds from 0.05 to 0.95. Since the default threshold of 0.5 may not yield the best precision-recall trade-off under severe class imbalance, optimal thresholds were identified independently per variant. Table 1 reports results at each variant's optimal F1 threshold,

providing a balanced trade-off between fraud detection rate and false-positive rate. Table 2 reports all variants at the standardised threshold of 0.5 for fair cross-variant comparison.

Table 1: Per-Variant Classification Metrics at Optimal Threshold

	Threshold	Precision	Recall	F1	G-Mean
V1	0.45	0.8763	0.7870	0.8293	0.8871
V2A	0.30	0.8440	0.8519	0.8479	0.9228
V2B	0.30	0.8667	0.8426	0.8545	0.9178
V3	0.90	0.6967	0.7870	0.7391	0.8869

Table 2: Cross-Variant Metrics at Threshold 0.5

Metric	V1	V2A	V2B	V3
Precision	0.8737	0.8830	0.8723	0.2217
Recall	0.7685	0.7685	0.7593	0.8889
F1	0.8177	0.8218	0.8119	0.3549
G-Mean	0.8766	0.8766	0.8713	0.9400
AUC-ROC	0.9767	0.9570	0.9522	0.9846
AUPRC	0.7509	0.8407	0.8311	0.7305
FP	13	11	12	337
Training Time	51.57s	53.47s	20.61s	29.10s

At threshold 0.5, V1, V2A, and V2B achieve comparable F1 scores, suggesting that once minority samples are reasonably distributed, the specific partitioning strategy has limited impact on threshold-dependent performance. The more telling distinction lies in AUPRC, a threshold-independent metric better suited to imbalanced data. Stratified repartitioning in V2A and V2B guarantees approximately 49 minority samples per partition, ensuring each partition runs meaningful SMOTE and produces well-calibrated trees. V1's random repartitioning provides no such guarantee, degrading SMOTE quality and hurting ranking performance across thresholds even when F1 at 0.5 appears competitive.

V3 tells a different story. Its global broadcast design achieves the highest recall, G-Mean, and AUC-ROC, as every tree is trained on the full diversity of fraud patterns. However, this comes at a severe cost of 337 false positives at threshold 0.5, compared to just 11 to 13 for local variants. In fraud detection, false positives translate directly to incorrectly blocked legitimate transactions, making V3 operationally impractical at the default threshold despite its strong discrimination ability. This highlights a fundamental tension between local and global parallelisation strategies: local variants constrain SMOTE synthesis to partition-local minority samples, producing well-calibrated ensembles, while V3 maximises minority visibility at the cost of calibration.

Regarding training time, V2B is the fastest overall at 20.61s, demonstrating that a single targeted stratified shuffle is sufficient for guaranteed minority distribution without the

cost of a global sort. Among local variants, V1 records 51.57s from a single random repartition shuffle, while V2A records the highest runtime at 53.47s due to its two-shuffle stratified sort. Notably, V1 and V2B both incur a single shuffle, yet V2B's targeted stratified approach delivers substantially better minority distribution and AUPRC at lower cost, highlighting that shuffle design matters as much as shuffle count. V3 records 29.10s as the global variant, as its broadcast design avoids redistributing the majority class entirely, shipping only the 90 KB minority set directly to workers. V2B is therefore selected as the representative stratified variant for the scalability experiments in Sections B and C, alongside V1 and V3.

B. Partition Sensitivity

Prior to the cross-variant comparison, Approaches A and B were used to develop experimental rigour. Approach A varied partition count while keeping a fixed number of bags per partition, meaning total ensemble size grew alongside partition count, confounding the two effects. Approach B addressed this by fixing the total trees at 80 and adjusting bags per partition, more cleanly isolating the partition count. Approach C, used for the cross-variant comparison here, further fixes the majority and minority bag size per tree so that partition count is the sole variable. All three variants were tested at partition counts of 2, 4, 8, and 16.

V1 achieved the strongest runtime scaling, falling from 21.97s at 2 partitions to 2.45s at 16, yielding a maximum speed-up of 8.97x and approaching ideal linear scaling. V2B reduced from 58.26s to 15.32s, a 3.80x speed-up, with its higher baseline reflecting the upfront cost of stratified repartitioning. V3 ranged from 39.77s to 11.14s, achieving only 3.34x speed-up, as the minority broadcast represents a fixed overhead that does not diminish with additional partitions.

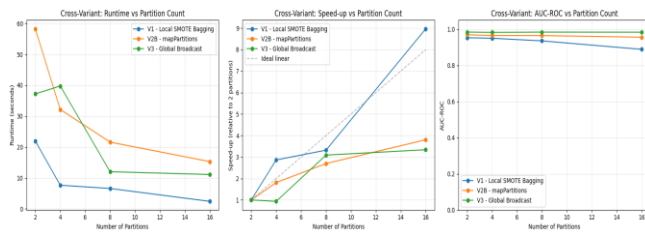


Figure 4: Partition Sensitivity Comparison

However, AUC-ROC results reveal the cost of V1's scalability as shown in Figure 4. Its AUC dropped by 0.0643 across partition counts (0.9529 to 0.8887), as larger partition counts leave fewer minority samples per partition, degrading SMOTE neighbourhood quality. V2B was considerably more stable with a drop of only 0.0136, as stratified repartitioning maintains consistent minority representation regardless of partition count. V3 was the most stable, with a negligible drop of 0.0019, since every worker retains access to the full minority pool irrespective of partitioning. These results indicate a clear trade-off: V1 scales most efficiently but at the cost of classification stability; V3 maintains the most

consistent predictive quality but scales least effectively; V2B offers a reasonable balance between the two.

C. Dataset Size Scaling

The experiment measures how the training runtime and classification performance change as data volume grows which was sub-sampled at 25%, 50%, 75%, and 100%, with both classes sampled proportionally to preserve the imbalance ratio at every level. This was done to prevent class-ratio effects from confounding the size effect. All other hyperparameters are held fixed (8 partitions, T=10 bags, K=5, max depth=10).

Results are summarised in Figure 5. All three variants exhibit sub-linear runtime growth across a 4x increase in data: V1 scales at 3.14x (8.53s to 26.77s), with near-zero repartition cost at all sizes (0.32s - 1.04s via coalesce). V2B scales at 1.19x (27.46s to 32.71s), though this understates the true scaling cost. The global shuffle dominates total time at small sizes (56% of runtime at 25%), compressing the apparent ratio as data grows. V3 scales at 3.45x (8.68s to 29.96s), with broadcast size growing linearly from 22.7 KB to 87.0 KB as minority samples increase with negligible overhead. However, V3 shows an anomalous spike at 75% data (50.15s), likely due to memory pressure at that intermediate partition workload, which warrants caution in interpreting its overall scaling behaviour.

AUC-ROC is stable across all data fractions for every variant with the ranges of 0.014 (V1), 0.018 (V2B), and 0.010 (V3). This confirms that none of the distributed approximations degrades classification quality as data volume increases. AUPRC is consistently higher for V2B and V3 than V1, reflecting the benefit of guaranteed or globally-aware minority neighbourhood coverage during SMOTE synthesis.

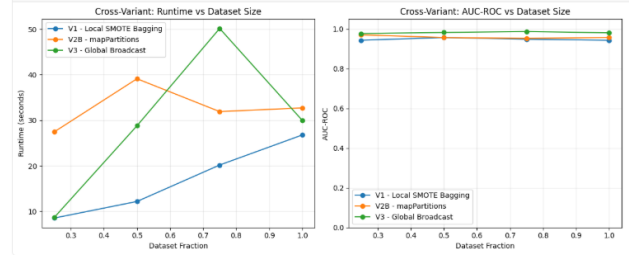


Figure 5: Dataset Size Scaling Comparison

The key discriminating factor across variants is therefore not performance stability but repartitioning overhead. V1 introduces a single upfront partition to establish balanced partitions, V2B pays a fixed shuffle overhead for guaranteed balance and V3 eliminates shuffle via broadcast, achieving the highest AUC but at the cost of less predictable runtime.

V. DISCUSSIONS

The results reveal a clear trade-off between communication overhead and SMOTE synthesis quality. V1 introduces a single upfront shuffle to distribute data across partitions after which all the operations are entirely local. This achieves the strongest partition speedup - 8.97x - but is the most sensitive

to partition composition in the raw data. 5 of 8 partitions initially contained zero minority instances, making local SMOTE impossible without preprocessing. V2B resolves this via a single global shuffle, yielding consistently higher AUPRC (0.8311 vs 0.7509 for V1 at full scale) at the cost of variable shuffle overhead (5.69s to 24.39s). V3 eliminates the shuffle entirely via broadcast - 87 KB at full scale - and achieves the highest AUC-ROC - 0.9804, but requires threshold tuning as its F1 at the default 0.5 threshold is lower than V1 and V2B, which indicates less well-calibrated majority-vote scores.

The achieved AUC-ROC scores - V1: 0.9767, V2B: 0.9522, V3: 0.9804 - match or exceed published non-distributed baselines on the same dataset Dal Pozzolo et al. [1] reported 0.93 to 0.96 using SMOTE with random forests which confirms that the distributing SMOTE Bagging across Spark workers does not sacrifice classification quality. Key bottlenecks remain - the $O(n^2)$ local K-NN search in SMOTE becomes problematic at larger minority set sizes, tree fits within each partition are sequential using single CPU core, limiting within-partition speedup, and all predictions are collected to the driver for majority voting which would become a memory bottleneck at large test set scales. These structural limitations are inherent to the current design and would require approximate K-NN methods and distributed voting to fully resolve.

The principal limitation of the experimental study is dataset scale. All variants complete within under a minute, which limits the ability to observe scaling differences that would emerge at true big data volumes. Single-run timing measurements also introduce variability. The anomalous V3 spike at 75% data size (50.15s vs 29.96s at 100%) likely reflects cluster-side noise rather than a reproducible pattern. Finally, experiments varied logical partition count but not physical cluster size, leaving weak and strong scaling behaviour uncharacterised.

VI. CONCLUSION

Three distributed SMOTEBagging variants were designed, implemented, and evaluated in Apache Spark against a dataset with 0.17% minority class, built from scratch using PySpark's mapPartitions API to minimise data movement during oversampling.

All variants demonstrated sub-linear runtime growth (V1: 3.14 $\times$, V2B: 1.19 $\times$, V3: 3.45 $\times$ for a 4 $\times$ data increase), and AUC-ROC remained stable across all data fractions (ranges: 0.014, 0.018, 0.010), confirming that distributed approximations do not degrade classification quality with scale. V1 achieved the strongest partition speedup (8.97 $\times$) but showed the highest AUC sensitivity to partition count (range 0.064). V2B achieved moderate speedup (3.80 $\times$) with the highest AUPRC (0.8311 to 0.8407). V3 achieved the highest AUC-ROC (0.9768 to 0.9868) but exhibited diminishing speedup at higher partition counts and less predictable runtime.

No single variant dominates across all dimensions. Deployment guidance is therefore: V1 when minimising communication overhead is critical, V2B when reliable minority coverage per partition is required, and V3 when ranking quality takes precedence and the minority set remains small enough to broadcast efficiently.

Future work should explore approximate nearest-neighbour methods such as HNSW or FAISS to address the $O(n^2)$ k-NN bottleneck, and a distributed majority-vote mechanism using reduceByKey to eliminate the driver-side aggregation bottleneck. A full weak and strong scaling study varying both cluster resources and data volume would also provide a more complete picture of scalability.

VII. REFERENCES

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Comput. Intell. Data Mining (CIDM)*, Cape Town, South Africa, Dec. 2015, pp. 159–166.
- [2] M. Galar, A. Fernández, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 4, pp. 463–484, Jul. 2012.
- [3] S. Wang and X. Yao, "Diversity analysis on imbalanced data sets by using ensemble models," in *Proc. IEEE Symp. Comput. Intell. Data Mining (CIDM)*, Nashville, TN, USA, Mar. 2009, pp. 324–331.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [5] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, Aug. 1996.
- [6] Machine Learning Group – ULB, "Credit Card Fraud Detection," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [7] M. Juez-Gil, J. J. Rodríguez, C. García-Orsorio, and J. Arnaiz-González, "Approx-SMOTE: Fast SMOTE for big data on Apache Spark," *Neurocomputing*, vol. 464, pp. 432–437, Nov. 2021.
- [8] W. C. Sleeman and B. Krawczyk, "Multi-class imbalanced big data classification on Spark," *Knowledge-Based Systems*, vol. 212, p. 106598, Jan. 2021.
- [9] K. Sid and M. Batouche, "Distributed heterogeneous ensemble learning on Apache Spark for ligand-based virtual screening," *Int. J. Data Mining, Modelling and Management*, vol. 13, no. 1/2, pp. 160–191, 2021.